Instructivo de Práctica — Sesión 2: Widgets básicos y avanzados

> 📌 **Qué vas a entregar:** al terminar esta práctica vas a tener la primera versión navegable de ExploraEC: una lista real de lugares, su pantalla de detalle, un formulario para agregar lugares nuevos, y una barra de navegación inferior con 3 secciones. Guarda bien tu `PlaceCard` y el patrón de navegación de hoy: son la base de la **Tarea 1** (se asigna al cierre de esta sesión, se entrega en la Sesión 4).

Objetivo

Repasar Dart con ejercicios cortos, dejar el proyecto bajo control de versiones, practicar `setState` / `initState` / `dispose` con un widget aparte, y extender `exploraec` con el catálogo de widgets básicos y avanzados de Flutter: modelo de datos, tarjetas reutilizables, listas, navegación entre pantallas, formularios validados y una barra de navegación inferior.

Requisitos previos

- Haber completado la práctica de la Sesión 1: proyecto `exploraec` corriendo, con la pantalla de bienvenida propia.
- Entorno Flutter verificado (`flutter doctor` sin ✗).
- Cuenta de GitHub activa (crear gratis en github.com/join si no se tiene) — hoy se sube el proyecto por primera vez.

Tiempo estimado

Total: 60 minutos

Paso	Descripción	Tiempo
1	Dart y control de versiones	18 min
2	Traer el código de la Sesión 2	4 min
3	`ContadorDemo`: practicar `setState` / `initState` / `dispose`	6 min

Paso	Descripción	Tiempo
4	Modelo `Place` y datos de ejemplo	6 min
5	`PlaceCard` y la pantalla de Inicio (`ListView`)	6 min
6	Pantalla de Detalle y navegación	6 min
7	Formulario "Agregar lugar" con validación	8 min
8	Barra de navegación inferior	6 min

Paso 1 — Dart y control de versiones (18 min)

Por qué se hace

Antes de construir widgets propios, conviene practicar la sintaxis completa de Dart (variables, control de flujo, bucles, null safety, funciones con parámetros nombrados, colecciones y clases con herencia) en un archivo simple, ejecutable directamente desde la terminal — y dejar el proyecto de la Sesión 1 bajo control de versiones antes de que crezca.

Instrucciones

1. Crear la carpeta y el archivo:

```
mkdir -p lib/playground
```

Crear `lib/playground/dart_basics.dart` con este contenido exacto:

```
// ExploraEC — Sesión 2: fundamentos de Dart
// Ejecutar con: dart run lib/playground/dart_basics.dart

void main() {
  ejercicioVariables();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Control de flujo: pendiente (descomenta ejercicioControlDeFlujo) ---');
  // ejercicioControlDeFlujo();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Bucles: pendiente (descomenta ejercicioBucles) ---');
  // ejercicioBucles();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Null safety: pendiente (descomenta ejercicioNullSafety) ---');
  // ejercicioNullSafety();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Funciones: pendiente (descomenta ejercicioFunciones) ---');
  // ejercicioFunciones();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Colecciones: pendiente (descomenta ejercicioColecciones) ---');
  // ejercicioColecciones();

  // TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
  print('--- Clases y herencia: pendiente (descomenta ejercicioClases) ---');
  // ejercicioClases();
}

void ejercicioVariables() {
  var nombre = 'ExploraEC';
  final creado = DateTime.now();
  const version = '1.0.0';
  print('App: $nombre v$version — creado: $creado');
}

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
```

```
// void ejercicioControlDeFlujo() {
//   String categorizarDistancia(double metros) {
//     if (metros < 500) {
//       return 'Muy cerca';
//     } else if (metros < 2000) {
//       return 'Cerca';
//     } else {
//       return 'Lejos';
//     }
//   }
//
//   print(categorizarDistancia(300));
//   print(categorizarDistancia(1200));
//   print(categorizarDistancia(5000));
// }

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
// void ejercicioBucles() {
//   final nombresDeEjemplo = ['Parque Ejido', 'Café Central', 'Museo'];
//   for (final nombre in nombresDeEjemplo) {
//     print('Lugar: $nombre');
//   }
//
//   var contador = 0;
//   while (contador < 3) {
//     print('Vuelta número $contador');
//     contador++;
//   }
// }

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
// void ejercicioNullSafety() {
//   String? apodo;
//   print('Apodo antes de asignar: ${apodo ?? "(sin apodo)"}');
//   apodo = 'Explo';
//   print('Apodo ya asignado, longitud: ${apodo.length}');
// }

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
```

```
// void ejercicioFunciones() {
//   String saludar(String nombre, {String saludo = 'Hola', bool mayusculas = false})
{
//     final texto = '$saludo, $nombre';
//     return mayusculas ? texto.toUpperCase() : texto;
//   }
//
//   print(saludar('Ana'));
//   print(saludar('Ana', saludo: 'Bienvenida', mayusculas: true));
// }

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
// void ejercicioColecciones() {
//   final categorias = <String>{'Parque', 'Cafetería', 'Museo'};
//   print('Categorías únicas: $categorias');
//
//   final distancias = <String, double>{'Parque Ejido': 300, 'Museo': 1800};
//   print('Distancia al Museo: ${distancias['Museo']} m');
//   print('Distancia a un lugar inexistente: ${distancias['Playa']}');
//
//   final nombres = ['Café Central', 'Parque Ejido', 'Museo'];
//   final conCafe = nombres.where((n) => n.contains('Café')).toList();
//   print('Lugares con "Café": $conCafe');
// }

// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.
// void ejercicioClases() {
//   final ana = Persona('Ana', 28);
//   print(ana.presentarse());
//
//   final consola = Consola();
//   consola.notificar('Lugar agregado con éxito');
// }
//
// class Persona {
//   final String nombre;
//   final int edad;
//
//   Persona(this.nombre, this.edad);
```

```
//
//   String presentarse() => 'Soy $nombre y tengo $edad años';
// }
//
// abstract class Notificable {
//   void notificar(String mensaje);
// }
//
// class Consola extends Notificable {
//   @override
//   void notificar(String mensaje) => print('Aviso: $mensaje');
// }
```

2. Ejecutar y observar la salida:

```
dart run lib/playground/dart_basics.dart
```

3. Descomentar `ejercicioControlDeFlujo()` (borrar la línea `print('--- Control de flujo...')` y el `//` de la llamada y del bloque de la función) y volver a ejecutar. Repetir con `ejercicioBucles()`, `ejercicioNullSafety()`, `ejercicioFunciones()`, `ejercicioColecciones()` y `ejercicioClases()`, ejecutando después de cada una.
4. Inicializar el repositorio Git y subirlo a GitHub:

```
git init
git add .
git commit -m "Proyecto Flutter inicial de ExploraEC"
git branch -M main
```

`git branch -M main` renombra la rama actual a `main` sin crear una nueva — necesario porque `git init` no siempre crea la rama `main` por defecto (depende de si `init.defaultBranch` está configurado); sin este paso, el `push` de abajo falla si la rama quedó como `master`.

Crear un repositorio nuevo en GitHub (por ejemplo `exploraec`, igual que la carpeta local — evita confundirlo con `exploraec-app`, el repo de referencia de solo lectura del Paso 2) y conectar:

```
git remote add origin <URL-de-tu-repositorio>
git push -u origin main
```

5. Agregar el repo de referencia del curso como un segundo remoto de solo lectura — de aquí se trae el código nuevo de cada sesión, sin volver a clonarlo aparte:

```
git remote add starter https://github.com/Patricio-CEDIA/exploraec-app.git
git fetch starter
```

Resultado esperado

`dart run lib/playground/dart_basics.dart` imprime en consola los 6 ejercicios sin errores. `git log` muestra al menos un commit, visible también en GitHub.

Errores comunes

- **`Target of URI doesn't exist: 'package:flutter/material.dart'`** al ejecutar `dart run` sobre un archivo que sí importa Flutter. *Solución:* `lib/playground/dart_basics.dart` no importa `material.dart` a propósito — es Dart puro, ejecutable sin el motor de Flutter; si aparece este error es que se copió mal el archivo.
- **Dejar dos bloques activos a la vez** (el `print` de "pendiente" y la llamada real sin comentar). *Solución:* borrar siempre la línea del `print` de aviso antes de descomentar el bloque real correspondiente.
- **La terminal se congela y no vuelve a mostrar el símbolo de espera** al descomentar `ejercicioBucles()`. *Solución:* revisa que el `while (contador < 3)` tenga, dentro de su cuerpo, la línea `contador++;` — sin ella, la condición nunca deja de cumplirse.
- **Subir la carpeta `build/` a GitHub**. *Solución:* `flutter create` ya genera un `.gitignore` que la excluye — si igual aparece en `git status`, confirmar que no se corrió `git add -f`.
- **`error: src refspec main no concuerda con ninguno`** al hacer `git push -u origin main`. *Solución:* falta `git branch -M main` — sin `init.defaultBranch` configurado, `git init` deja la rama como `master`, no `main`; confirma con `git branch` cuál quedó activa.

✅ Antes de continuar, confirma que...

- ☐ `dart run lib/playground/dart_basics.dart` corre sin errores, con los 6 ejercicios descomentados.
- ☐ El repositorio existe en GitHub con el commit inicial.

- ☐ `git remote -v` muestra tanto `origin` (tu repositorio) como `starter` (el repo de referencia del curso).
- ☐ `git branch` muestra `main` como rama activa (no `master`).

Paso 2 — Traer el código de la Sesión 2 (4 min)

Por qué se hace

El repo `starter` trae, por rama, el código ya resuelto de cada sesión — traer solo `lib/`, `pubspec.yaml` y `PLACEHOLDERS.md` de la rama de hoy evita copiar archivos a mano y dejar alguno afuera por error. Como `lib/main.dart` existe tanto en tu rama actual (la pantalla de bienvenida con estilo de Stitch de la Sesión 1) como en `starter/sesion-02`, este `checkout` lo reemplaza igual que al resto de `lib/` — es el único archivo que habías modificado hasta ahora, y es justamente el que este paso trae listo para la Sesión 2.

Instrucciones

1. Traer los archivos de la rama `sesion-02` sobre tu propio proyecto — esto reemplaza tu `lib/main.dart` actual (la pantalla de Stitch de la Sesión 1) por la versión provisional de la Sesión 2, y agrega el resto de `lib/` (`models/`, `widgets/`, `screens/`, `playground/`) más `pubspec.yaml`:

```
git fetch starter
git checkout starter/sesion-02 -- lib pubspec.yaml PLACEHOLDERS.md
flutter pub get
```

2. Abrir `PLACEHOLDERS.md` (recién llegado a la raíz de tu proyecto) — es la guía de qué archivo tiene qué bloque comentado y en qué paso se descomenta, con un comentario `// Por qué:` en el código explicando cada uno.
3. Abrir `lib/main.dart` y ubicar el bloque `// TODO(sesion-02)` cerca de la barra de navegación inferior — todavía no lo toques, es para el Paso 8.

Resultado esperado

`flutter run` ya no muestra la pantalla de bienvenida de Stitch, sino la pantalla de Inicio de ExploraEC (con "Cargando lugares..." y sin datos todavía — normal, el Paso 4 los agrega). `lib/playground/contador_demo.dart`, `lib/playground/contador_demo_main.dart`, `lib/models/place.dart`, `lib/widgets/place_card.dart`, `lib/screens/home_screen.dart`, `lib/`

`screens/detail_screen.dart`, `lib/screens/add_place_screen.dart`, `lib/screens/map_placeholder_screen.dart` y `lib/screens/favorites_placeholder_screen.dart` existen dentro de tu proyecto, y `PLACEHOLDERS.md` está en la raíz.

Errores comunes

- **`error: pathspec 'starter/sesion-02' did not match any file(s) known to git`**. *Solución:* falta el `git fetch starter` antes del `checkout`, o el remoto `starter` no quedó bien agregado en el Paso 1 — revisa con `git remote -v`.
- **`Target of URI doesn't exist: 'package:exploraec/screens/home_screen.dart'`**. *Solución:* confirma que el `checkout` terminó sin errores y que `lib/` tiene las subcarpetas `models`, `widgets`, `screens`, `playground`.
- **"¿A dónde se fue mi pantalla de bienvenida de Stitch?"**. Es esperado: el `checkout` de este paso reemplaza `lib/main.dart` a propósito. La pantalla de Stitch cumplió su rol en la Sesión 1 (practicar el flujo de diseño con MCP); de aquí en adelante `lib/main.dart` es la app de ExploraEC que se construye durante el resto del curso.

✅ Antes de continuar, confirma que...

- ☐ `lib/models`, `lib/widgets`, `lib/screens` y `lib/playground` existen dentro de tu proyecto.
- ☐ `PLACEHOLDERS.md` está en la raíz del proyecto.
- ☐ `flutter pub get` corrió sin errores.

Paso 3 — `ContadorDemo`: practicar `setState` / `initState` / `dispose` (6 min)

Por qué se hace

Antes de aplicar `StatefulWidget` dentro de ExploraEC (la barra de navegación inferior, Paso 8), conviene sentir `setState()`, `initState()` y `dispose()` en un widget chico y aislado, con la teoría todavía fresca — sin la fricción extra del resto de la app alrededor.

Instrucciones

1. Ejecutar `ContadorDemo` con su punto de entrada alternativo, sin tocar `lib/main.dart` (el de ExploraEC recién traído en el Paso 2):

```
flutter run -t lib/playground/contador_demo_main.dart
```

2. Mirar la terminal: apenas arranca, aparece `initState: el contador arrancó en 0`.
3. Tocar el botón `+` un par de veces. El número en pantalla se queda en 0, pero la terminal sí muestra el mensaje de `_incrementar()` cada vez — la app "sabe" que cambió, pero nadie le avisó a Flutter que debía redibujar.
4. Abrir `lib/playground/contador_demo.dart`, localizar el `// TODO(sesion-02)` dentro de `_incrementar()`. Borrar la línea `print('Tocaste +, pero setState()...')` y descomentar el bloque `setState() { _veces++; }`.
5. Guardar y tocar `+` de nuevo: ahora el número sí cambia en pantalla.
6. Hacer hot reload (tecla `r` en la terminal) con el contador en un número mayor a 0, y confirmar que el número se conserva.
7. Hacer hot restart (tecla `R` mayúscula) y confirmar dos cosas: el número vuelve a 0, y `initState: el contador arrancó en 0` se imprime de nuevo en la terminal — la prueba de que hot restart crea un `State` completamente nuevo.
8. Cerrar `ContadorDemo` (`q` en la terminal) y volver a `lib/main.dart` con:

```
flutter run
```

Resultado esperado

Antes de descomentar el `setState()`, el número se queda fijo en 0 pese a tocar el botón. Después de descomentarlo, el número aumenta con cada toque, se conserva en hot reload, y vuelve a 0 (con `initState()` ejecutándose de nuevo) en hot restart.

Errores comunes

- **El número sigue sin cambiar después de descomentar el bloque**. *Solución:* revisa que se haya borrado la línea `print('Tocaste +, pero setState()...')` — si ambos bloques quedan activos, Dart marca error de código duplicado dentro de la función.
- **`Target of URI doesn't exist: 'contador_demo.dart'`** al correr `flutter run -t ...`. *Solución:* confirma que el Paso 2 trajo la carpeta `lib/playground/` completa — revisa con `ls lib/playground`.
- **`flutter run` (sin `-t`) sigue mostrando ContadorDemo en vez de ExploraEC**. *Solución:* sin `-t`, Flutter siempre usa `lib/main.dart` por defecto — si aparece ContadorDemo, es que la terminal donde corre sigue siendo la del comando anterior; ciérrala (`q`) y volvé a correr `flutter run` en una terminal nueva o la misma ya liberada.

✅ Antes de continuar, confirma que...

- ☐ Antes de descomentar el `setState()`, el número se queda en 0 pese a tocar `+`.
- ☐ Después de descomentarlo, el número aumenta con cada toque.
- ☐ Un hot restart reinicia el número a 0 y vuelve a imprimir `initState(...)` en la terminal.

Paso 4 — Modelo `Place` y datos de ejemplo (6 min)

Por qué se hace

Antes de dibujar cualquier lista, hace falta el molde de datos que la describe — la misma idea de clases vista en esta sesión, aplicada ahora a algo que sí se muestra en pantalla.

Instrucciones

1. Abrir `lib/models/place.dart` — la clase `Place` ya está completa y activa (no tiene `TODO`).
2. Ubicar el bloque `// TODO(sesion-02): borra la línea de abajo y descomenta el bloque completo.` sobre la lista `lugaresEjemplo`.
3. Borrar la línea `final List<Place> lugaresEjemplo = [];` (la lista vacía provisional) y descomentar el bloque completo de 6-8 lugares de ejemplo.

Resultado esperado

`lugaresEjemplo` contiene ahora una lista de objetos `Place` con nombres reales de lugares (parques, cafeterías, museos) en vez de una lista vacía.

Errores comunes

- **Dejar ambos bloques activos** (la lista vacía Y la descomentada): Dart marca error de "The name 'lugaresEjemplo' is already defined". *Solución:* borrar por completo la línea provisional antes de descomentar.

✅ Antes de continuar, confirma que...

- ☐ `lugaresEjemplo` tiene 6 u 8 elementos, no está vacía.
- ☐ El proyecto compila sin errores (`flutter analyze` o simplemente dejar correr `flutter run` en paralelo).

Paso 5 — `PlaceCard` y la pantalla de Inicio (6 min)

Por qué se hace

Con los datos listos, corresponde construir la pieza visual que los representa (`PlaceCard`) y la pantalla que los lista (`HomeScreen`) — exactamente el patrón `ListView.builder` + widget reutilizable visto en la teoría.

Instrucciones

1. Abrir `lib/widgets/place_card.dart`. Localizar el `// TODO(sesion-02)` marcado "(Paso 5 — cuerpo de la tarjeta)", sobre el `child` del `InkWell`.
2. Borrar el `Padding` con `Text('Cargando tarjeta...')` provisional y descomentar el `Padding` con el `Row` completo (ícono de categoría + `Column` con nombre y descripción truncada).
3. Abrir `lib/screens/home_screen.dart`. Localizar el `// TODO(sesion-02)` sobre el cuerpo del `Scaffold`.
4. Borrar el `Center(child: Text('Cargando lugares...'))` provisional y descomentar el `ListView.builder` que recorre `lugaresEjemplo`.
5. Ejecutar (o hot reload si ya estaba corriendo):

```
flutter run
```

Resultado esperado

La pantalla de Inicio muestra 6-8 tarjetas, cada una con el nombre y la categoría de un lugar, desplazables verticalmente.

Errores comunes

- **`A RenderFlex overflowed by N pixels`**. *Solución:* revisa que el `Text` del nombre dentro de `PlaceCard` esté envuelto en `Expanded` dentro de su `Row` — nombres largos sin `Expanded` desbordan.
- **La lista aparece vacía aunque `lugaresEjemplo` tiene datos**. *Solución:* confirma que `itemCount: lugaresEjemplo.length` (no un número fijo) y que `itemBuilder` usa el `index` recibido, no un índice fijo como `0`.

✅ Antes de continuar, confirma que...

- ☐ La pantalla de Inicio muestra las tarjetas de los lugares de ejemplo.
- ☐ Cada tarjeta muestra nombre y categoría, sin desbordes visuales.

Paso 6 — Pantalla de Detalle y navegación (6 min)

Por qué se hace

Ver un lugar en la lista no alcanza — cada tarjeta debe llevar a una pantalla con la información completa, aplicando `Navigator.push` tal como se vio en la teoría.

Instrucciones

1. Abrir `lib/widgets/place_card.dart` de nuevo. Localizar el segundo `// TODO(sesion-02)`, sobre el `onTap` del `Card`.
2. Borrar el `onTap: null,` provisional y descomentar el `onTap` real, que hace `Navigator.push` hacia `DetailScreen`.
3. `lib/screens/detail_screen.dart` ya está completo, sin `TODO` — ábrelo para revisar cómo recibe el `Place` por su constructor.
4. Probar: tocar una tarjeta de la lista y confirmar que se abre el Detalle con la información completa; usar el botón de retroceso y confirmar que vuelve a Inicio con la lista intacta.

Resultado esperado

Tocar cualquier tarjeta navega al Detalle de ese lugar específico (no siempre el mismo); volver atrás regresa a la lista sin recargarla.

Errores comunes

- **`Navigator operation requested with a context that does not include a Navigator`**. *Solución:* confirma que el `onTap` está dentro del `build()` de un widget que ya vive dentro de `MaterialApp` — no debería ocurrir si solo se descomentó el bloque indicado, pero es la causa típica si se movió código de lugar.
- **Todas las tarjetas abren el mismo lugar**. *Solución:* confirma que `DetailScreen(place: place)` usa la variable `place` del `itemBuilder` (el lugar de esa fila específica), no una variable fija.

✅ Antes de continuar, confirma que...

- ☐ Cada tarjeta abre el Detalle del lugar correcto.
- ☐ Volver atrás regresa a la lista de Inicio sin errores.

Paso 7 — Formulario "Agregar lugar" con validación (8 min)

Por qué se hace

Hasta este paso la app solo lee datos fijos; el formulario agrega la posibilidad de capturar datos nuevos del usuario, validados antes de aceptarlos — la otra mitad de cualquier app real.

Instrucciones

1. Abrir `lib/screens/add_place_screen.dart`. Localizar el `// TODO(sesion-02)` sobre los 3 `TextFormField` (nombre, categoría, descripción).
2. Borrar los `TextField` provisionales (sin validación) y descomentar los `TextFormField` completos, cada uno con su `validator`.
3. Localizar el segundo `// TODO(sesion-02)`, sobre el `onPressed` del botón "Guardar".
4. Borrar el `onPressed: null,` provisional y descomentar el bloque que llama a `_formKey.currentState!.validate()` y, si es válido, agrega el nuevo `Place` a `lugaresEjemplo` y hace `Navigator.pop`.
5. Navegar al formulario tocando el botón flotante `+` de la esquina inferior derecha de Inicio (ya viene activo en `HomeScreen`) y probar: - Presionar "Guardar" con los campos vacíos → deben aparecer mensajes de error, sin cerrar el formulario. - Completar los 3 campos y presionar "Guardar" → el formulario se cierra y el nuevo lugar aparece en la lista de Inicio.

Resultado esperado

Un formulario vacío no se guarda y muestra sus errores; un formulario completo agrega un lugar nuevo, visible de inmediato en la lista de Inicio.

Errores comunes

- **`type 'Null' is not a subtype of type 'String'`**. *Solución:* revisa que cada `TextEditingController` se haya creado (`TextEditingController()`) y no se esté leyendo `.text` de un controller no inicializado.
- **El botón "Guardar" no hace nada**. *Solución:* confirma que se descomentó el `onPressed` completo y se borró la línea `onPressed: null,` — con ambos presentes, Dart usaría el último definido, pero es más fácil de detectar si no queda ninguno duplicado.
- **El formulario se cierra aunque un campo esté vacío**. *Solución:* revisa que ese campo tenga su `validator` descomentado — un campo sin `validator` nunca bloquea el guardado.

✅ Antes de continuar, confirma que...

- ☐ Enviar el formulario vacío muestra los 3 mensajes de error.
- ☐ Completar el formulario agrega el lugar a la lista de Inicio y cierra la pantalla.

Paso 8 — Barra de navegación inferior (6 min)

Por qué se hace

ExploraEC necesita moverse entre secciones del mismo nivel (Inicio, Mapa, Favoritos) sin apilarlas como el Detalle — el patrón de `BottomNavigationBar` visto en la teoría, la primera aplicación real de `StatefulWidget` / `setState()` dentro de ExploraEC (ya practicados en el Paso 3 con `ContadorDemo`).

Instrucciones

1. Abrir `lib/main.dart`. Localizar el `// TODO(sesion-02)` sobre el `bottomNavigationBar` del `Scaffold` raíz.
2. Borrar el `bottomNavigationBar: null,` provisional y descomentar el `BottomNavigationBar` completo con sus 3 ítems (Inicio, Mapa, Favoritos).
3. Localizar el segundo `// TODO(sesion-02)`, sobre el cuerpo que decide qué pantalla mostrar según `_indiceActual`.
4. Borrar el `body: const HomeScreen(),` fijo provisional y descomentar el `body` que alterna entre `HomeScreen`, `MapPlaceholderScreen` y `FavoritesPlaceholderScreen` según `_indiceActual`.
5. Probar tocando cada una de las 3 pestañas.
6. Confirmar todo con Git:

```
git add .
git commit -m "Sesion 2: widgets basicos y avanzados en ExploraEC"
git push
```

Resultado esperado

Tocar cada pestaña de la barra inferior cambia el contenido de la pantalla entre la lista de lugares, el placeholder de Mapa ("Próximamente: mapa real (Sesión 4)") y el placeholder de Favoritos ("Próximamente: favoritos (Sesión 7)").

Errores comunes

- **La pestaña se resalta pero el contenido no cambia**. *Solución:* confirma que el `body` del `Scaffold` usa `_indiceActual` con un `switch` o `if` que devuelve un widget distinto por índice — no un `body` fijo.
- **`setState() called after dispose()`**. *Solución:* poco común en este paso, pero si aparece, revisa que no quede ningún `Future`/temporizador pendiente de una pantalla que ya se cerró — la misma pareja `initState`/`dispose` practicada en el Paso 3.

✅ Antes de continuar, confirma que...

- ☐ Las 3 pestañas cambian correctamente el contenido visible.
- ☐ `git push` se completó sin errores.

Entregable de la práctica

- `lib/playground/dart_basics.dart` con los 6 ejercicios descomentados y ejecutados (control de flujo, bucles, null safety, funciones, colecciones, clases con herencia).
- `lib/playground/contador_demo.dart` con `setState()` activo, probado con hot reload y hot restart.
- Repositorio en GitHub con el commit inicial.
- Pantalla de Inicio con la lista real de lugares de ejemplo.
- Navegación funcional hacia el Detalle de cualquier lugar.
- Formulario "Agregar lugar" validado y funcional.
- Barra de navegación inferior con sus 3 secciones.
- Repositorio actualizado en GitHub.

Conexión con la siguiente sesión

En la Sesión 3 se aplica un tema Material consistente a toda la app, se agregan estados de carga/vacío/error reutilizables (hoy la lista siempre tiene datos; pronto no será así) y se decide entre `ListView`/`GridView` según el ancho de pantalla. Todo se construye sobre las mismas pantallas de hoy — es indispensable llegar con Inicio, Detalle, Formulario y la navegación inferior funcionando.